

Contents of lecture 3:

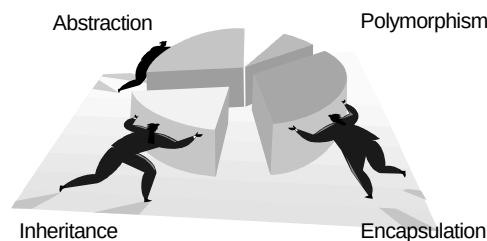
1. More Object Oriented Programming

- Inheritance
- Polymorphism

Principles of Object Oriented Programming: Part 2: Inheritance and Polymorphism

Inheritance and polymorphism: Simple ideas

In the previous lecture the notion of abstraction and encapsulation was explored (the first half of the A-PIE mnemonic). The remaining two fundamental OOP concepts of inheritance and polymorphism are explored in this lecture.



Fortunately both concepts are reasonably straightforward.

Inheritance

In a sentence, inheritance involves building upon an existing class so that additional or more-specialised functionality is added.

A key design aspect of inheritance is that it is used in a manner which is understandable and supported by a clear, justifiable relationship.



To give an example of *bad* design, in the Deitel & Deitel book a Shape class is defined which is inherited by Point, which in turn is inherited by Circle. Finally, Circle is inherited by Cylinder. The Point class contained a number of methods for setting/retrieving the x and y coordinates. For reasons of convenience, the Circle class inherited these methods from Point and added further methods for setting/retrieving the radius.

The Circle class should not inherit from Point as such a relationship is not supported by normal shape categorisation (a circle cannot be viewed as a more specialised form of point – instead it would be more accurate to say that a circle can be defined in terms of a central point and a radius).

Inheritance should be used to capture hierarchical relationships integral to the problem being modelled. As such, any use of inheritance should be justifiable in terms of the relationship that is being modelled.

Inheritance and Java

Just as inheritance in real life is ‘what you get from your parents’ in OOP, inheritance is ‘what you get from your parent class’ (in this case, the data items and functionality of the class that is inherited from).

Note, only those methods/data that are declared as public or protected will be inherited. Constructors, destructors and anything declared as private will not be inherited, although the constructors of the inherited class can still be invoked, and Java will ensure any destructors are correctly called.

In Java every class has a parent class (the exception to this is the `java.lang.Object` class, which is the ultimate parent of all classes). Within Java, the parent class is either named explicitly or provided implicitly by the Java VM, for example:

```
class Boss extends Employee { ... } ← Explicitly naming parent  
or  
class Employee { ... } ← Implicit naming.  
Interpreted as: class Employee extends Object
```

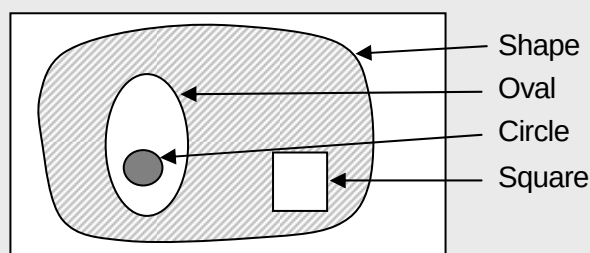
Superclass and subclass terminology

The terms superclass and subclass are used extensively within OOP. A definition of each term follows:

- **Superclass:** a *parent* or *base* class. Superclass wrongly suggests that the parent class has more functionality than the subclass. Generally a subclass is a more specialised form of the superclass. The term is drawn from set theory.
- **Subclass:** A *child* class that inherits from, or extends, a superclass. As with superclasses, the term is taken from set theory.

Aside: Set theory

Consider the following Venn diagram:



The diagram shows the ‘Shape’, ‘Oval’, ‘Circle’ and ‘Square’ classes. A class is a superclass if it encompasses any other classes (e.g. the Oval class is a superclass as it contains the Circle class). A class is a subclass if it is encompassed within any other class, e.g. Oval is also a subclass (of the Shape superclass).

Example of inheritance within Java

An example of inheritance in Java now follows. Firstly an `Employee` class is defined, which is extended by `Programmer/SalesPerson` classes.

As can be seen the class offers methods for setting and getting the name and monthly pay of the employee. A method to determine the yearly pay is also included.

```
public class Employee
{
    private String name;
    private float fMonthlyPay;

    public void setMonthlyPay( float fMonthlyPay )
    { this.fMonthlyPay = fMonthlyPay; }

    public float getMonthlyPay()
    { return fMonthlyPay; }

    public void setName( String name )
    { this.name = name.toString(); }

    public String getName()
    { return name; }

    public float getYearlyPay()
    { return fMonthlyPay * 12.0f; }
}
```

A `Programmer` class is defined, which extends the `Employee` class by introducing a 'project' field:

```
public class Programmer extends Employee
{
    private String project;

    public void setProject( String project )
    { this.project = project.toString(); };

    public String getProject()
    { return project; }
}
```

Also a `Salesperson` class is defined, as follows:

```
public class SalesPerson extends Employee
{
    private float fBonus;

    public void setBonus( float fBonus )
    { this.fBonus = fBonus; }

    public float getBonus()
    { return fBonus; }

    public float getYearlyPay()
    { return getMonthlyPay() * 12.0f + fBonus; }
}
```

Note, in order to determine the yearly pay it is necessary to call `getMonthlyPay` as `fMonthlyPay` is declared to be private in the `Employee` class.

Given the above, we can obtain the following:

The above illustrates that superclass methods can be invoked from a subclass object (i.e. Programmer class calling setName, getYearlyPay, etc. of the Employee class), provided the subclass object does not *override* the method (in which case the subclass method is called, i.e. SalesPerson calls the version of getYearlyPay() defined within the SalesPerson class).

```
SalesPerson bob = new SalesPerson();
bob.setName( "Bob" );
bob.setMonthlyPay( 1200.0f );
bob.setBonus( 2500.0f );
bob.getYearlyPay();
```

Gives 16900

```
Programmer susan = new Programmer();
susan.setName( "Susan" );
susan.setMonthlyPay( 1750.0f );
susan.setProject( "Billing Update" );
susan.getYearlyPay();
```

Gives 21000

```
Employee mystery = new Employee();
mystery.getYearlyPay();
```

Gives 0

This is an example of a *naming collision*, whereby if a variable/method in the subclass has the same name as a variable/method in the superclass, then it supersedes, or *hides*, the superclass. In general, variables and methods should only be hidden if the subclass replaces the functionality of the superclass.

Note that the superclass variables and methods are still available to the subclass, even if they are hidden by the subclass (assuming they have public or protected access), e.g.:

```
public float getYearlyPay()
{ return getMonthlyPay() * 12.0f + fBonus; }
```

Could also have been written as:

```
public float getYearlyPay()
{ return super.getYearlyPay() + fBonus; }
```

A subclass can also be a superclass

Using inheritance, the following class based manipulation is valid:

```
Employee employees[3];
employees[0] = bob;
employees[1] = susan;
employees[2] = mystery;

for( int idx = 0; idx < employees.length; idx++ )
    System.out.println( "\nEmployee "
        + (idx+1) + employees[idx].getName() );
```

Which prints: "1 Bob", "2 Susan", "3 "

Hence, any subclass object can be treated as an instance of the corresponding superclass object. This stands to reason, as a subclass can do everything a superclass object can do. The ability to treat a subclass object as an instance of the superclass is a key principle of OOP. As such, it provides a means of generically processing classes. Hence, we can write a method that operates on Employee objects, and as a consequence of inheritance, any object that extends Employee can safely be passed to the method.



Improving the Employee example

The `Employee` class defined above can be improved through the use of protected data and abstract classes, as follows:

Protected data

Methods or data declared to be **protected** can only be accessed as follows:

- From within the class in which they are defined.
- From any subclass that inherits the class in which they are defined.

In general, protected data access can be used in those situations where subclass performance is important, or to make available housekeeping methods, etc. that should not be available outside of the scope of the class hierarchy.

As such, **protected** offers an intermediate level of privacy between **public** and **private**. Hence, we might wish to change the `Employee` class, so that the class data is defined to be protected, as follows:

```
public class Employee
{
    protected String name;
    protected float fMonthlyPay;
    ...
}
```

The `getYearlyPay` of the `SalesPerson` class can now be written as follows:

```
public float getYearlyPay()
{ return fMonthlyPay * 12.0f + fBonus; }
```

Abstract keyword

At times it is desirable to create a class of which no instances will ever directly exist (i.e. it is impossible to instantiate an instance of the class) – such a class is defined as abstract.

The `Employee` class is an example of such a class. No objects of type `Employee` should ever exist, as employees of the company will be objects of type `Programmer`, `Boss`, `SalesPerson`, etc. Hence, the sole purpose of an **abstract** class is to provide an interface which other classes can inherit and implement (i.e. all employees of a company share a certain number of characteristics in common, which can be encapsulated within the `Employee` class). Classes from which instances can be instantiated are called **concrete** classes.

As there should never be an instance of an `Employee` object we should define the class as being abstract, as follows:

```
public abstract class Employee
{ ... }
```

Defined as such, no instances of the `Employee` class can be declared.

Aside: Methods within a class can also be declared to be abstract (thereby making the entire class abstract).

Any subclass that inherits from an abstract superclass normally provides implementations of all superclass methods defined to be abstract. If it does not, then the subclass is itself abstract, and no instances can be declared. Note that an abstract class can contain concrete methods and data definitions.

Although it is not possible to create objects of an abstract superclass, it is permissible to declare references to an abstract superclass, and thereby allow subclasses to be processed generically, e.g. for the employee class:

Assume: public abstract class Employee { ... }

```
Employee employees[2];  
employees[0] = bob;  
employees[1] = susan;  
// employees[2] = mystery;
```

**This is now invalid as
mystery cannot be
instantiated.**

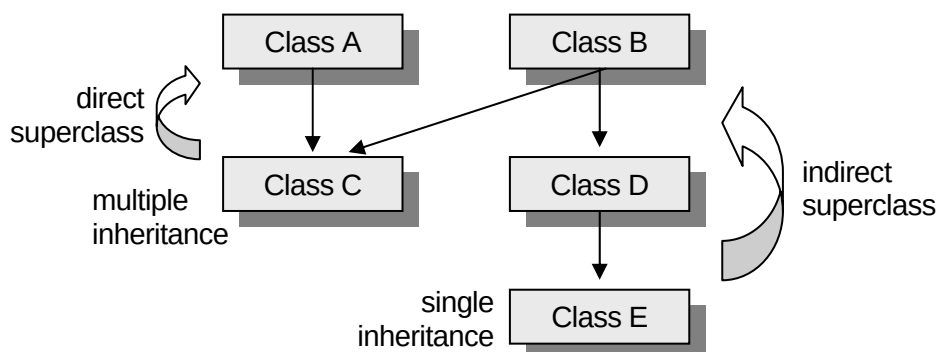
```
for( int idx = 0; idx < employees.length; idx++ )  
    System.out.println( "\nEmployee "  
        + (idx+1) + employees[idx].getName() );
```

Which prints: "1 Bob", "2 Susan"

In many respects, the abstract keyword does not add any new functionality; rather it restricts what the programmer can do with the class. Primarily this is of benefit to the class designer who can ensure that the class is not used inappropriately.

Different types of Inheritance

Consider the following class hierarchy:



The terminology is defined below:

- The *direct superclass* of a subclass is that class from which it explicitly inherits within the class definition, e.g. class c extends a $\leftarrow$ a is a direct superclass of c.
- An *indirect superclass* of a subclass is a class which the superclass inherits from (directly or indirectly), e.g. class e extends d; class d extends b $\leftarrow$ b is an indirect superclass of e.
- *Single inheritance* entails that a class can only inherit directly from one superclass
- *Multiple inheritance* entails that a class can inherit directly from a number of superclasses (not supported by Java)

Multiple inheritance entails that a class has more than one parent (i.e. inherits the methods, data, etc. of several different classes).

Multiple inheritance is much less commonly required when modeling problems when compared to single inheritance. This is fortunate, as languages that support multiple inheritance, e.g. C++, suffer from several headaches concerning data and method access.

These problems can be solved, however, in order to do so a sizeable and complex set of language specifications must be followed.



Java does not support multiple inheritance

Java sidesteps the difficulties associated with multiple inheritance by simply supporting only single inheritance. Instead, the use of interfaces is advocated as a means of unproblematically modelling many complex relationships (interfaces are explored later).

Casting subclasses and superclasses

It was previously noted that a subclass object could be treated as an instance of the relevant superclass (which makes perfect sense). In this section, we are going to look at this relationship more carefully:

```
Class Mammal {...}
Class Dog extends Mammal {...}
Class Cat extends Mammal {...}

Dog fido = new Dog();
Dog rusty = new Cat();
Mammal anAnimal;
```

As we have seen, it is always possible to make a more general class (superclass) refer to a specialised subclass, i.e. a dog can be considered as a mammal: `anAnimal = fido;`

The reverse of this is not always true, i.e. a mammal may not be a dog. In this case, Java will only permit the conversion if it is explicitly cast: `fido = (Dog)anAnimal;`



Casting can be generalised as follows:

- You can always make a parent reference (superclass) point to a child class (subclass). A cast is not needed, nor is the operation dangerous. If necessary you can assign several levels up an inheritance hierarchy (i.e. to a grandparent class, etc.)
- You can attempt to assign a parent reference (superclass) to a child reference (subclass) through the use of a cast, i.e. `child = (child)parent`. The assignment is checked at run time, and if it is invalid then the exception `ClassCastException` is thrown.
- It is impossible to assign or cast between arbitrary, unrelated classes, e.g. Dog can never be assigned to Cat, etc.

```
superclass = subclass
// always valid

subclass=(subclass)superclass
// valid at compile time, checked at runtime

subclass = superclass
// not valid at compile time, needs a cast

someClass = someUnrelatedClass
somcClass = (someClass)someUnrelatedClass
// not valid, won't compile
```

Polymorphism

Although the word ‘polymorphism’ may sound complicated, it actually refers to a straightforward concept. The word originates from the Greek language, and means ‘many shapes’. Within object oriented languages it is used to denote one name referring to several different methods.

Within Java there are really two different types of polymorphism: **overloading** and **overriding**. Both of which are considered below:

Overloading

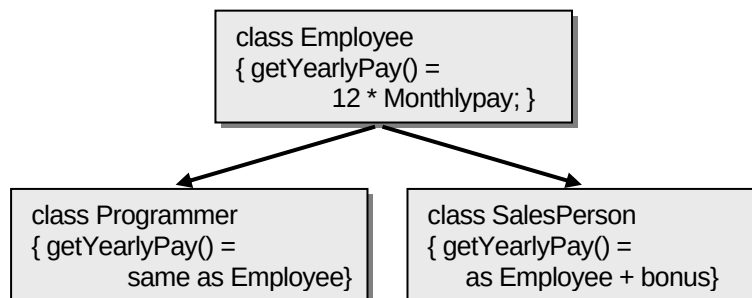
Overloading is something you’ve already encountered. It simply refers to the ability to define several methods to have the same name within a particular class (noting that the methods must have different signatures so that the Java VM can work out which method to call).

For example:

```
long Math.max( long a, long b)
int Math.max( int a, int b )
double Math.max( double a, double b )
float Math.max( float a, float b )
```

Overriding

This is the second, more complex, form of polymorphism (and normally is what people mean when they use the term polymorphism). Overriding, as previously mentioned, occurs whenever a subclass has a method with the same signature (number, type and order of parameters) as a method in one of its superclasses. When this happens, the method in the derived class overrides the method in the superclass. Let us consider again the Employee/SalesPerson/Programmer class hierarchy as previously defined.



As can be seen, the `Employee` class defines a `getYearlyPay()` method, which is inherited by both the `Programmer` and `SalesPerson` classes. However, the `SalesPerson` class modifies the method by adding a yearly bonus.

Given the following code:

```
SalesPerson sue = new SalesPerson();
sue.setMonthlyPay( 1700.Of );
sue.setBonus( 3000.Of );

Employee aPerson = sue;
System.out.println( "Pay = " +
    aPerson.getYearlyPay() );
```

What will be the output? This is potentially unclear as 'aPerson' is a reference to an Employee object, suggesting that the Employee.getYearlyPay() method will be called. However, 'aPerson' really points to 'sue', hence the correct method to call should really be SalesPerson.getYearlyPay().

In Java, and other languages that support polymorphism, the SalesPerson.getYearlyPay() method will be called. Hence, polymorphism guarantees the following:

If a subclass object is assigned to a superclass reference, and a method of the superclass is invoked which is overridden in the subclass object, then via polymorphism, the correct method of the subclass object will be called.

i.e. if sue is assigned to aPerson, then aPerson.getYearlyPay() will actually call sue.getYearlyPay();

Polymorphism is something free

Note that polymorphism is not something the programmer has to actively introduce (unlike making classes abstract, etc.). Rather, it is something provided by the language for free. The Java VM, at run-time, will automatically work out what method should be called.

Given this, the programmer need only be aware that Java provides this feature, so that it can be used to their advantage.

Why is polymorphism useful?

With polymorphism it is possible to write a method that *correctly* processes lots of different types of classes (all within a class hierarchy), through a superclass reference (ensuring that the correct subclass methods will be called):

Without polymorphism

```
For each employee
{
    If Programmer
        then Calc Programmer pay
    If SalesPerson
        then Calc SalesPerson pay
    If Boss
        then Calc Boss pay
}
```

With polymorphism

```
For each employee
{
    Calc Employee pay
}
```

Aside: References and object types

Java accomplishes polymorphic overriding thanks to each object carrying around a little extra bit of information about their type and the characteristics of that type. Using this, the Java VM can determine what data type a reference really points to. The extra bit of type information is also used when determining if a reference can be assigned to a particular type. For example, consider the code opposite:

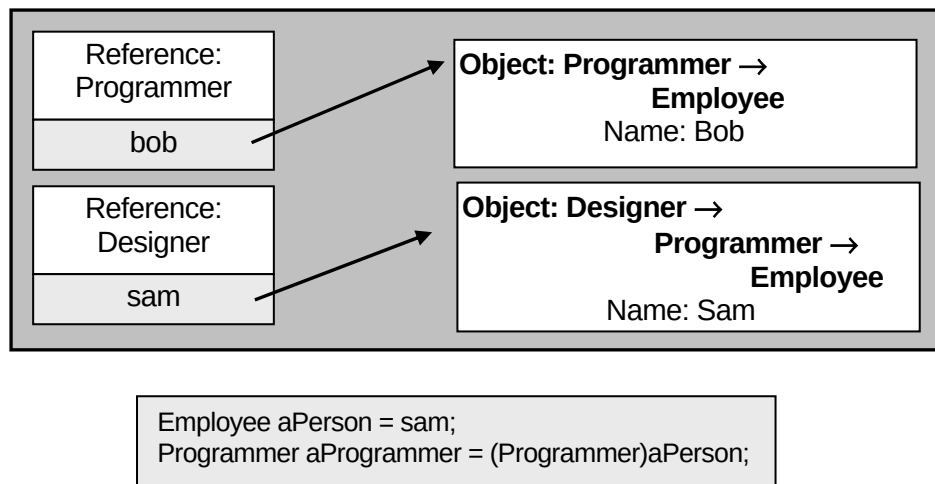
Class definitions:

```
Abstract Class Employee {...}
Class SalesPerson extends Employee {...}
Class Programmer extends Employee {...}
Class Designer extends Programmer {...}
```

Code:

```
Programmer bob = new Programmer();
Designer sam = new Designer();
```

which is realised as follows:



will check the 'sam' object to see if it can be considered as an Employee object (compile time). At runtime, the `aProgrammer = aPerson` will be checked to see if the object that `aPerson` really refers to (i.e. `sam`) can be considered as an instance of a `Programmer` (which it can).

Inheritance/polymorphism design benefits

Inheritance and polymorphism are useful from the following design perspectives:

- Ideal for representing hierarchical relationships (both in terms of ease of coding and code maintenance).
- Provides a powerful means through which more specialised classes may be introduced.
- When combined with polymorphism, inheritance provides a means of straightforwardly processing a wide range of different, but related, classes.
- Inheritance preserves the object-oriented principle of least privilege. This is achieved by limiting the data items and methods that are accessible to the subclass to those that are declared public or protected. A subclass cannot access private member/data of the superclass.

In fact, Java is structured in such a way that it offers the programmer a number of classes that can be extended to provide more specific functionality (i.e. no need to reinvent the wheel, the creators of Java intended their classes to be extended and built upon).

The 'is a' and 'has a' relationships

Sometimes in object-oriented design, problems may arise concerning the relationship between objects. In particular, should one class extend another class through inheritance, or alternatively include that class as a data item (e.g. should we have `class a extends b`, or `class a { b binstance; ... }`)?

These problems can be easily resolved through the 'is a' and the 'has a' questions. For example, 'a car *has an* engine' is true (container) whilst 'a car *is a* engine' is false (not inheritance). Also 'a dog *has a* mammal' is false (not container) whilst 'a dog *is a* mammal' is true (inheritance). Or, 'Tom Smith *is a* homo sapien *is a* mammal *is a* creature *is a* physical thing'.

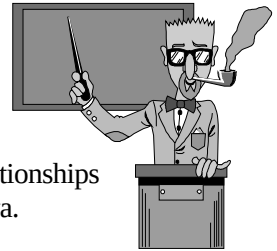
Practical 3

After this lecture you should explore the third practical pack which should enable you to investigate the material in this lecture.

Learning Outcomes

Once you have explored and reflected upon the material presented within this lecture and the practical pack, you should:

- Be able to define what constitutes inheritance within object-oriented programming and understand the broad principles of how inheritance is intended to be used from a design perspective.
- Understanding the terminology used to express/model hierarchical class relationships and also know how hierarchical class relationships can be created within Java.
- Understand from a design perspective the notion of polymorphism within object-oriented programming.
- Know how polymorphism is implemented within Java and be able to write programs that exploit polymorphism to provide generic class processing.



More comprehensive details can be found in the CSC735 Learning Outcomes document.